

Algorithms Project: Distinct Elements

1. Introduction

The objective of this project is to determine the number of unique values within a given zero-indexed array A consisting of N integers. This is a fundamental problem in data processing and algorithmic analysis.

2. Problem Constraints

- Number of elements (N): $[0..100,000]$
- Element range: $[-1,000,000..1,000,000]$

3. Algorithm 1: Sorting Approach (Non-Recursive)

3.1 Strategy

This algorithm uses a nested-loop sorting mechanism (Bubble Sort) to organize the array. Once sorted, identical elements are adjacent. By iterating through the array and comparing each element with its predecessor, the algorithm counts the number of distinct values.

3.2 Pseudo-code

```
ALGORITHM solution_sorting(A):
    n = length of A

    IF n IS 0 THEN
        RETURN 0

    FOR i FROM 0 TO n - 1 DO:
        FOR j FROM 0 TO n - i - 2 DO:
            IF A[j] > A[j + 1] THEN:
                temp = A[j]
                A[j] = A[j + 1]
                A[j + 1] = temp
            END IF
        END FOR
    END FOR
```

```

distinct_count = 1

FOR i FROM 1 TO n - 1 DO:
    IF A[i] IS NOT EQUAL TO A[i - 1] THEN:
        distinct_count = distinct_count + 1
    END IF
END FOR

RETURN distinct_count
END ALGORITHM

```

3.3 Complexity Analysis

- Time Complexity: $O(N^2)$ due to the nested loops in the sorting phase.
- Space Complexity: $O(1)$ as the sorting is done in-place.

4. Algorithm 2: Recursive Approach

4.1 Strategy

This approach uses the "Divide and Conquer" principle. It takes the first element of the array, filters out all occurrences of that element from the rest of the list, and then recursively calls itself to count the remaining unique values.

4.2 Pseudo-code

```

Function solution_recursive(A):
    If A is empty: Return 0
    Set pivot = A[0]
    Create 'rest' = list of elements in A[1:] not equal to pivot
    Return 1 + solution_recursive(rest)

```

4.3 Complexity Analysis

- Time Complexity: $O(N^2)$ in the worst case (all elements distinct) due to repeated filtering.
- Space Complexity: $O(N)$ due to the recursion stack depth.

5. Comparison of Algorithms

Feature	Sorting Approach (Bubble)	Recursive Approach
Type	Non-recursive	Recursive
Time Complexity	$O(N^2)$	$O(N^2)$
Space Complexity	$O(1)$	$O(N)$
Best Usage	Simple Implementation	Educational Purposes

6. Implementation (Python)

```
def solution_sorting(A):
    n = len(A)
    if n == 0: return 0

    # Bubble Sort Phase
    for i in range(n):
        for j in range(0, n - i - 1):
            if A[j] > A[j + 1]:
                A[j], A[j + 1] = A[j + 1], A[j]

    # Counting Phase
    distinct_count = 1
    for i in range(1, n):
        if A[i] != A[i-1]:
            distinct_count += 1
    return distinct_count

def solution_recursive(A):
    if not A: return 0
    first = A[0]
    rest = [x for x in A[1:] if x != first]
    return 1 + solution_recursive(rest)
```